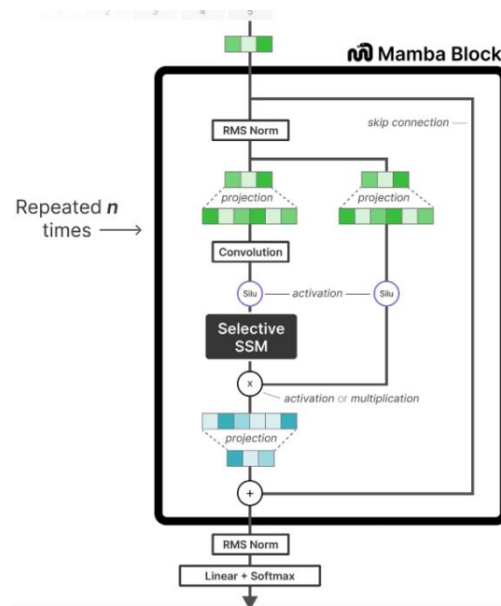


優化過程：

架構主要參考 Mamba 論文中的架構實作，將原 Transformer 中的 encoder block 換成 mamba block，主要流程是先建立一個 mamba block 之後再加入 normalization 和額外 penalty 的機制：



1. 建立 Mamba block：

Selective SSM(S6)：將原 SSM(Structured State Space Sequence

Model, S4)改造成其參數(A, B, C, delta)會隨著輸入的變化而影響，進

而達成模型的靈活性(模型能根據輸入內容選擇性將其記住或忽略)。

Algorithm 1 SSM (S4)

Input: $x : (B, L, D)$
Output: $y : (B, L, D)$
1: $A : (D, N) \leftarrow \text{Parameter}$
 ▷ Represents structured $N \times N$ matrix
2: $B : (D, N) \leftarrow \text{Parameter}$
3: $C : (D, N) \leftarrow \text{Parameter}$
4: $\Delta : (D) \leftarrow \tau_{\Delta}(\text{Parameter})$
5: $\overline{A}, \overline{B} : (D, N) \leftarrow \text{discretize}(\Delta, A, B)$
6: $y \leftarrow \text{SSM}(\overline{A}, \overline{B}, C)(x)$
 ▷ Time-invariant: recurrence or convolution
7: **return** y

Algorithm 2 SSM + Selection (S6)

Input: $x : (B, L, D)$
Output: $y : (B, L, D)$
1: $A : (D, N) \leftarrow \text{Parameter}$
 ▷ Represents structured $N \times N$ matrix
2: $B : (B, L, N) \leftarrow s_B(x)$
3: $C : (B, L, N) \leftarrow s_C(x)$
4: $\Delta : (B, L, D) \leftarrow \tau_{\Delta}(\text{Parameter} + s_{\Delta}(x))$
5: $\overline{A}, \overline{B} : (B, L, D, N) \leftarrow \text{discretize}(\Delta, A, B)$
6: $y \leftarrow \text{SSM}(\overline{A}, \overline{B}, C)(x)$
 ▷ Time-varying: recurrence (scan) only
7: **return** y

i. B, C, delta 參數：

```
# projects x to -dependent Δ, B, C
self.x_proj = nn.Linear(config.d_inner, config.dt_rank + 2 * config.d_state, bias=False)
```

```
delta, B, C = torch.split(deltaBC,
                             [self.config.dt_rank, self.config.d_state, self.config.d_state], dim=-1)
delta = F.softplus(self.dt_proj(delta))
```

$$s_B(x) = \text{Linear}_N(x), s_C(x) = \text{Linear}_N(x),$$

$$s_\Delta(x) = \text{Broadcast}_D(\text{Linear}_1(x)), \text{ and } \tau_\Delta = \text{softplus},$$

1: $A : (D, N) \leftarrow \text{Parameter}$

▷ Represents structured $N \times N$ matrix

2: $B : (B, L, N) \leftarrow s_B(x)$

3: $C : (B, L, N) \leftarrow s_C(x)$

4: $\Delta : (B, L, D) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$

ii. 離散化(1a→2a)：將 A, B 離散化成 $\bar{A}$, $\bar{B}$ 。

```
deltaA = torch.exp(delta.unsqueeze(-1) * A)
deltaB = delta.unsqueeze(-1) * B.unsqueeze(2)
```

$$\bar{A} = \exp(\Delta A) \quad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$$

$$h'(t) = Ah(t) + Bx(t) \quad (1a) \quad h_t = \bar{A}h_{t-1} + \bar{B}x_t \quad (2a)$$

$$y(t) = Ch(t) \quad (1b) \quad y_t = Ch_t \quad (2b)$$

iii. 序列掃描(S6 運算)：主要時做的部分為(方程式 2a, 2b 的部

```
def selective_scan_seq(self, x, delta, A, B, C):
    _, L, _ = x.shape
    Bx = B * (x.unsqueeze(-1)) # (B, L, N, N)

    h = torch.zeros(x.size(0), self.d_model, self.d_model, device=A.device)

    hs = []
    for t in range(0, L):
        h = A[:, t] * h + Bx[:, t]
        hs.append(h)
    hs = torch.stack(hs, dim=1) # (B, L, N, N)

    y = (hs @ C.unsqueeze(-1)).squeeze(3) # (B, L, N, N) @ (B, L, N, 1) -> (B, L, N)
    return y
```

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t \quad (2a)$$

$$y_t = Ch_t \quad (2b)$$

分)

iv. 其他組件：Convolution Layer、RMS Normalization

Layer、Silu activation、Projction(Fully Connected Layer)

一開始的 Mamba 模型效能非常的差→

```
[最高, 最低, 開盤, 收盤]
result: tensor([[[11320.7773, -7889.2363, 5672.5889, 836.5897]]],
              grad_fn=<SqueezeBackward1>)
target: tensor([[[777., 753., 771., 753.]]])
val loss: 7243.82568359375
推論時間: 0.0390017032623291 seconds
Total Parameters: 27410
Trainable Parameters: 27410
```

2. 輸入模型前先做 **normalization**：為了縮小資料的數值，期望模型能加速收斂和提升穩定性(推論時間降低，RMS 也降低)。

```
[最高, 最低, 開盤, 收盤]  
result: tensor([[890.4307, 744.5018, 837.3010, 886.2328]],  
target: tensor([[777., 753., 771., 753.]])  
val loss: 93.65568542480469  
推論時間: 0.03769397735595703 seconds  
Total Parameters: 27410  
Trainable Parameters: 27410
```

3. 額外懲罰(延續 2)：加入一些期望模型能預測的方向，如未達成 loss 會增加。(例：價格不能小於 0、最高價不能低於其他價格、最低價不能高於其他價格...等)。效果反而更差

```
[最高, 最低, 開盤, 收盤]  
result: tensor([[123.0863, 104.4584, 71.4742, 128.2468]], grad_fn=<SqueezeBackward1>)  
target: tensor([[777., 753., 771., 753.]])  
val loss: 657.2408447265625  
推論時間: 0.03602743148803711 seconds  
Total Parameters: 27410  
Trainable Parameters: 27410
```

執行方法

1. 打開終端機進入 model 資料夾中
2. 輸入：`bash start.sh`
3. 開始執行